
Project Libre Document

Curiosity

By

Bobbali Manoj Kumar (24CSB1A09)

Borra Pardhiv (24CSB1A10)

Chilakala Bharat Teja Reddy(24CSB1A11)

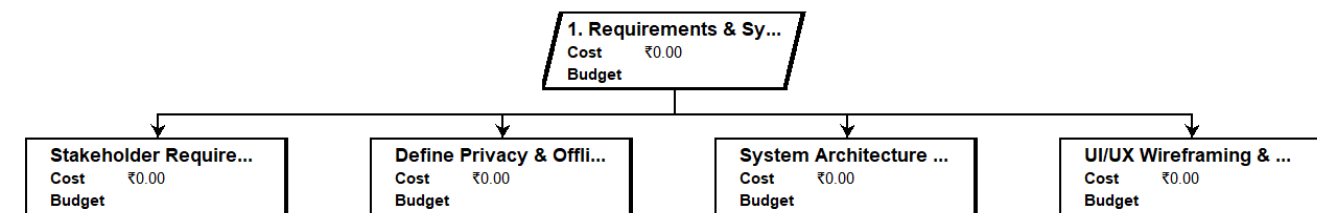
Part 1: Work Breakdown Structure (WBS)

1.1 Overview

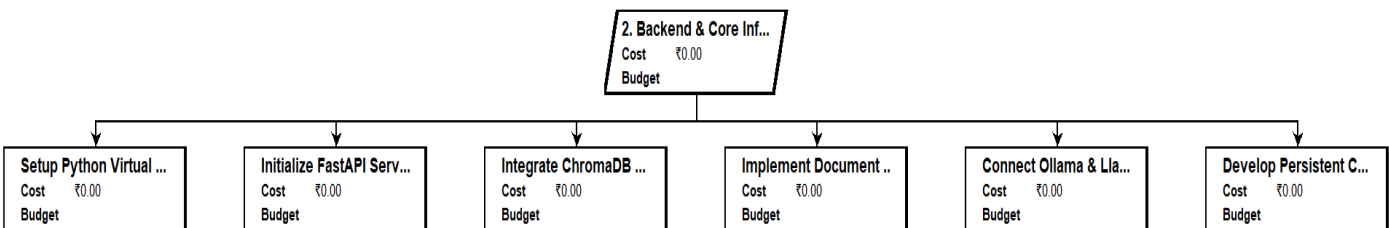
The Work Breakdown Structure (WBS) for the Air Gap AI project is meticulously organized into five major development phases. Each phase is further broken down into highly specific, manageable sub-tasks with associated costs, budgets, and resource allocations. This hierarchical breakdown serves as the foundational blueprint for the entire project, demonstrating clear project organization, distinct boundaries of scope, and a strategic roadmap from initial conception to final deployment. By establishing this WBS, the team ensures that no critical functionality—especially regarding the strict offline privacy requirements—is overlooked during the development lifecycle.

1.2 Major Project Phases

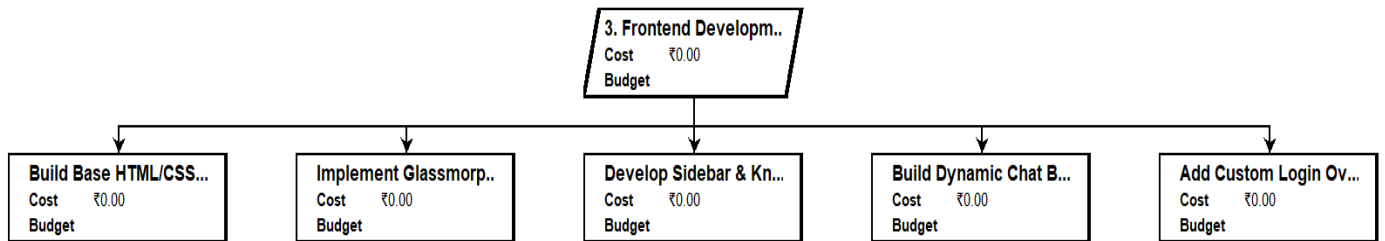
1) Requirements & System Design: The initial phase is dedicated to gathering detailed functional and non-functional requirements, specifically focusing on the critical offline privacy constraints required by Air Gap AI. This phase also includes the architectural planning of the system and the creation of UI/UX wireframes to establish the visual language.



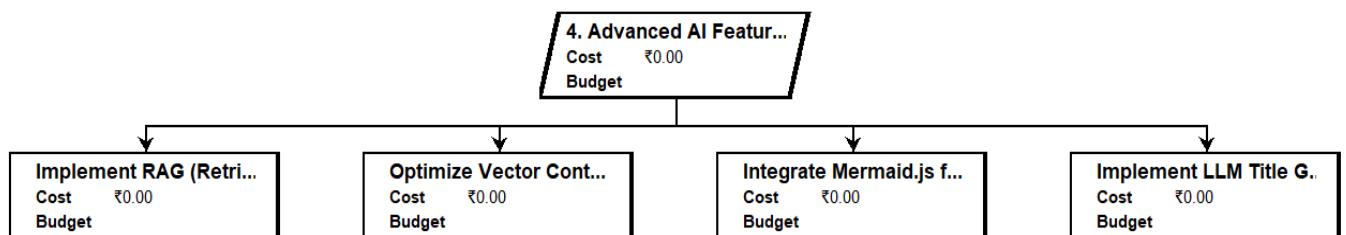
2) Backend & Core Infrastructure: This phase focuses on the underlying technical engine of Air Gap AI. It involves setting up the Python environment, building the Fast API routing system, establishing the Chroma DB vector database for local storage, and integrating the open-source Ollama models (Llama 3.2) required for offline inference.



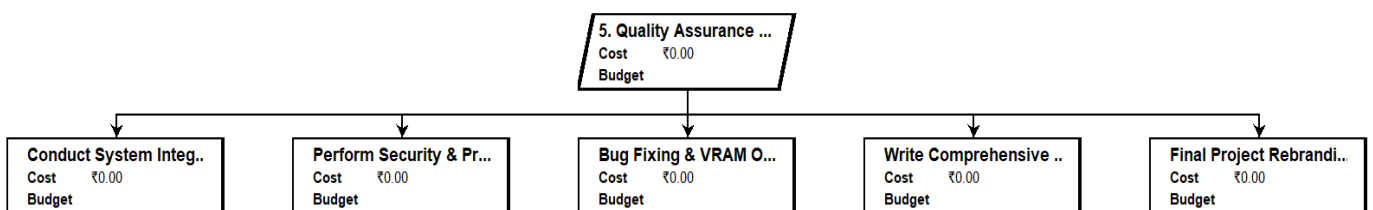
3) Frontend Development: The frontend phase transforms the wireframes into a fully functional, interactive user interface. This involves building the base layout and implementing a complex glass morphism design system to give the application a modern, sleek, and highly responsive user experience.



4) Advanced AI Features: This is the most complex phase of the project, focusing on integrating the Retrieval-Augmented Generation (RAG) pipeline. This phase ensures the AI can accurately pull information from local documents, optimize context windows, and dynamically generate Mermaid.js flowcharts based on user queries.



5) Quality Assurance & Deployment: The final phase is dedicated to rigorous testing and deployment. A massive focus is placed on a Security & Privacy Audit to ensure the system truly remains "air-gapped" with no external data leaks. It also involves bug fixing, VRAM optimization for lower-end hardware, and final code commits.



Resource allocation

Resource ID	Resource					
1	Lead AI Architect					
Task ID	Task	Work	Assignment Units	Assignment	Start	Finish
5	System Architecture & Data Flow	36 hours	100%	0 days	27/08/26, 8:00 am	02/09/26, 1:00 pm
21	Implement RAG (Retrieval-	44 hours	100%	0 days	22/10/26, 8:00 am	29/10/26, 1:00 pm
22	Optimize Vector Context Window	20 hours	100%	0 days	29/10/26, 8:00 am	02/11/26, 1:00 pm
12	Connect Ollama & Llama 3.2	20 hours	100%	0 days	24/09/26, 8:00 am	28/09/26, 1:00 pm
4	Define Privacy & Offline	8 hours	100%	0 days	26/08/26, 8:00 am	26/08/26, 5:00 pm
		128 hours				

Table 1.1 Resource Allocation Lead AI Architect

2	Backend Engineer					
Task ID	Task	Work	Assignment Units	Assignment	Start	Finish
28	Bug Fixing & VRAM Optimization	20 hours	100%	0 days	23/11/26, 8:00 am	25/11/26, 1:00 pm
11	Implement Document Parsing	44 hours	100%	0 days	17/09/26, 8:00 am	24/09/26, 1:00 pm
10	Integrate ChromaDB Local	36 hours	100%	0 days	11/09/26, 8:00 am	17/09/26, 1:00 pm
9	Initialize FastAPI Server &	28 hours	100%	0 days	08/09/26, 8:00 am	11/09/26, 1:00 pm
12	Connect Ollama & Llama 3.2	20 hours	100%	0 days	24/09/26, 8:00 am	28/09/26, 1:00 pm
24	Implement LLM Title Generation	36 hours	100%	0 days	06/11/26, 8:00 am	12/11/26, 1:00 pm
13	Develop Persistent Chat History	28 hours	100%	0 days	28/09/26, 8:00 am	01/10/26, 1:00 pm
8	Setup Python Virtual	12 hours	100%	0 days	07/09/26, 8:00 am	08/09/26, 1:00 pm
		224 hours				

Table 1.2 Resource Allocation Backend Engineer

3	Frontend / UI Developer					
Task ID	Task	Work	Assignment Units	Assignment	Start	Finish
23	Integrate Mermaid.js for Dynamic	36 hours	100%	0 days	02/11/26, 8:00 am	06/11/26, 1:00 pm
17	Develop Sidebar & Knowledge	28 hours	100%	0 days	12/10/26, 8:00 am	15/10/26, 1:00 pm
15	Build Base HTML/CSS Layout	28 hours	100%	0 days	01/10/26, 8:00 am	06/10/26, 1:00 pm

Resource ID	Resource					
6	UI/UX Wireframing &	28 hours	100%	0 days	02/09/26, 8:00 am	07/09/26, 1:00 pm
18	Build Dynamic Chat Bubble	28 hours	100%	0 days	15/10/26, 8:00 am	20/10/26, 1:00 pm
16	Implement Glassmorphism	36 hours	100%	0 days	06/10/26, 8:00 am	12/10/26, 1:00 pm
19	Add Custom Login Overlay &	20 hours	100%	0 days	20/10/26, 8:00 am	22/10/26, 1:00 pm
		204 hours				

Table 1.3 Resource Allocation UI/UX Dev

4 QA / Security Engineer

Task ID	Task	Work	Assignment Units	Assignment	Start	Finish
28	Bug Fixing & VRAM Optimization	20 hours	100%	0 days	23/11/26, 8:00 am	25/11/26, 1:00 pm
27	Perform Security & Privacy Audit	28 hours	100%	0 days	18/11/26, 8:00 am	23/11/26, 1:00 pm
26	Conduct System Integration	36 hours	100%	0 days	12/11/26, 8:00 am	18/11/26, 1:00 pm
		84 hours				

Table1.4 Resource Allocation QA/Security Engineer

5 Project Manager

Task ID	Task	Work	Assignment Units	Assignment	Start	Finish
30	Final Project Rebranding & Git	20 hours	100%	0 days	27/11/26, 8:00 am	01/12/26, 1:00 pm
3	Stakeholder Requirements	16 hours	100%	0 days	24/08/26, 8:00 am	25/08/26, 5:00 pm
29	Write Comprehensive	20 hours	100%	0 days	25/11/26, 8:00 am	27/11/26, 1:00 pm
4	Define Privacy & Offline	8 hours	100%	0 days	26/08/26, 8:00 am	26/08/26, 5:00 pm
		64 hours				

Table1.5 Resource Allocation Project Manager

Part 2: Developing a Project Plan

The Air Gap AI project involves a vast array of different technical functionalities spanning from complex artificial intelligence logic to user interface design. To make the development process manageable, these functionalities are divided into sequential tasks and subtasks.

Activities, Durations, and Dependencies

		Name	Duration	Start	Finish	Predecessors	Resource Names
1		LocalLens AI Project	75 days	24/08/26, 8:00 am	04/12/26, 5:00 pm		
2		1. Requirements & System Design	10 days	24/08/26, 8:00 am	04/09/26, 5:00 pm		
3		Stakeholder Requirements Gathering	2 days	24/08/26, 8:00 am	25/08/26, 5:00 pm		Project Manager
4		Define Privacy & Offline Constraints	1 day	26/08/26, 8:00 am	26/08/26, 5:00 pm	3	Lead AI Architect;Project Manager
5		System Architecture & Data Flow Design	4.5 days	26/08/26, 12:00 pm	01/09/26, 5:00 pm	4	Lead AI Architect
6		UI/UX Wireframing & Glassmorphism Prototyping	3.5 days	01/09/26, 12:00 pm	04/09/26, 5:00 pm	5	Frontend / UI Developer
7		2. Backend & Core Infrastructure	18.5 days	04/09/26, 12:00 pm	30/09/26, 5:00 pm		
8		Setup Python Virtual Environment & Dependencies	1.5 days	04/09/26, 12:00 pm	07/09/26, 5:00 pm	6	Backend Engineer (Python)
9		Initialize FastAPI Server & Routing	3.5 days	07/09/26, 12:00 pm	10/09/26, 5:00 pm	8	Backend Engineer (Python)
10		Integrate ChromaDB Local Vector Storage	4.5 days	10/09/26, 12:00 pm	16/09/26, 5:00 pm	9	Backend Engineer (Python)
11		Implement Document Parsing (PDF, Word, OCR)	5.5 days	16/09/26, 12:00 pm	23/09/26, 5:00 pm	10	Backend Engineer (Python)
12		Connect Ollama & Llama 3.2 APIs	3 days	23/09/26, 12:00 pm	25/09/26, 5:00 pm	11	Backend Engineer (Python);Lead AI Architect
13		Develop Persistent Chat History (JSON)	3.5 days	25/09/26, 12:00 pm	30/09/26, 5:00 pm	12	Backend Engineer (Python)
14		3. Frontend Development	15.5 days	30/09/26, 12:00 pm	21/10/26, 5:00 pm		
15		Build Base HTML/CSS Layout	3.5 days	30/09/26, 12:00 pm	05/10/26, 5:00 pm	13	Frontend / UI Developer
16		Implement Glassmorphism Design System	4.5 days	05/10/26, 12:00 pm	09/10/26, 5:00 pm	15	Frontend / UI Developer
17		Develop Sidebar & Knowledge Base UI	3.5 days	09/10/26, 12:00 pm	14/10/26, 5:00 pm	16	Frontend / UI Developer
18		Build Dynamic Chat Bubble Interface	3.5 days	14/10/26, 12:00 pm	19/10/26, 5:00 pm	17	Frontend / UI Developer
19		Add Custom Login Overlay & Animations	2.5 days	19/10/26, 12:00 pm	21/10/26, 5:00 pm	18	Frontend / UI Developer
20		4. Advanced AI Features	15.5 days	21/10/26, 12:00 pm	11/11/26, 5:00 pm		
21		Implement RAG (Retrieval-Augmented Generation)	5.5 days	21/10/26, 12:00 pm	28/10/26, 5:00 pm	19	Lead AI Architect
22		Optimize Vector Context Window (k=12)	2.5 days	28/10/26, 12:00 pm	30/10/26, 5:00 pm	21	Lead AI Architect
23		Integrate Mermaid.js for Dynamic Flowcharts	4.5 days	30/10/26, 12:00 pm	05/11/26, 5:00 pm	22	Frontend / UI Developer
24		Implement LLM Title Generation Logic	4.5 days	05/11/26, 12:00 pm	11/11/26, 5:00 pm	23	Backend Engineer (Python)
25		5. Quality Assurance & Deployment	13.5 days	11/11/26, 12:00 pm	30/11/26, 5:00 pm		
26		Conduct System Integration Testing	4.5 days	11/11/26, 12:00 pm	17/11/26, 5:00 pm	24	QA / Security Engineer
27		Perform Security & Privacy Audit (AirGap Check)	3.5 days	17/11/26, 12:00 pm	20/11/26, 5:00 pm	26	QA / Security Engineer
28		Bug Fixing & VRAM Optimization	3 days	20/11/26, 12:00 pm	24/11/26, 5:00 pm	27	Backend Engineer (Python);QA / Security Engl...
29		Write Comprehensive Documentation (README)	2.5 days	24/11/26, 12:00 pm	26/11/26, 5:00 pm	28	Project Manager
30		Final Project Rebranding & Git Push	2.5 days	26/11/26, 12:00 pm	30/11/26, 5:00 pm	29	Project Manager

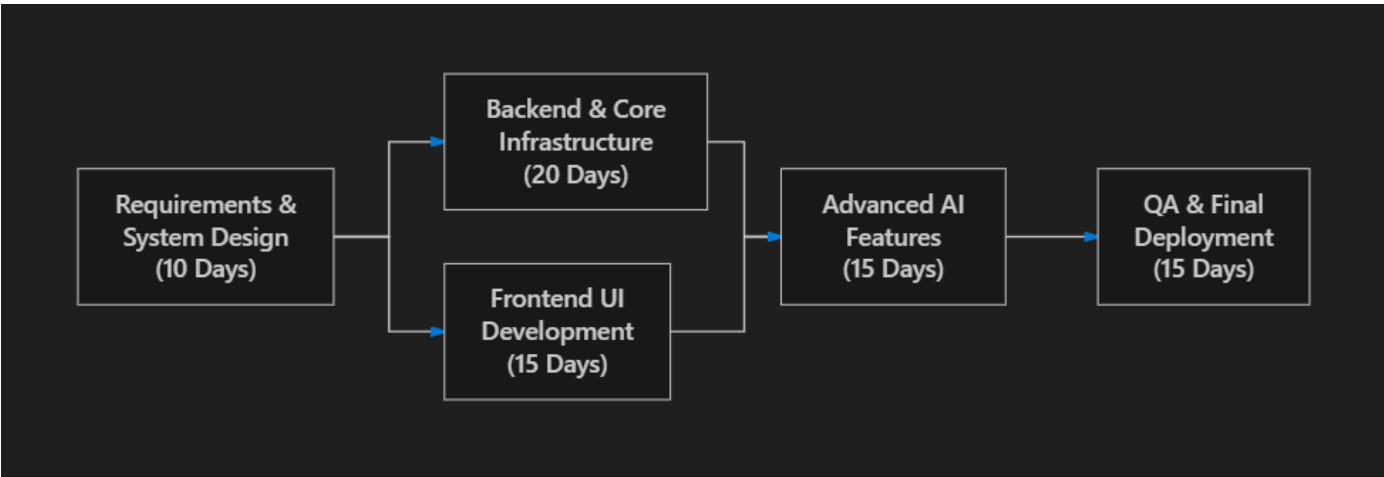
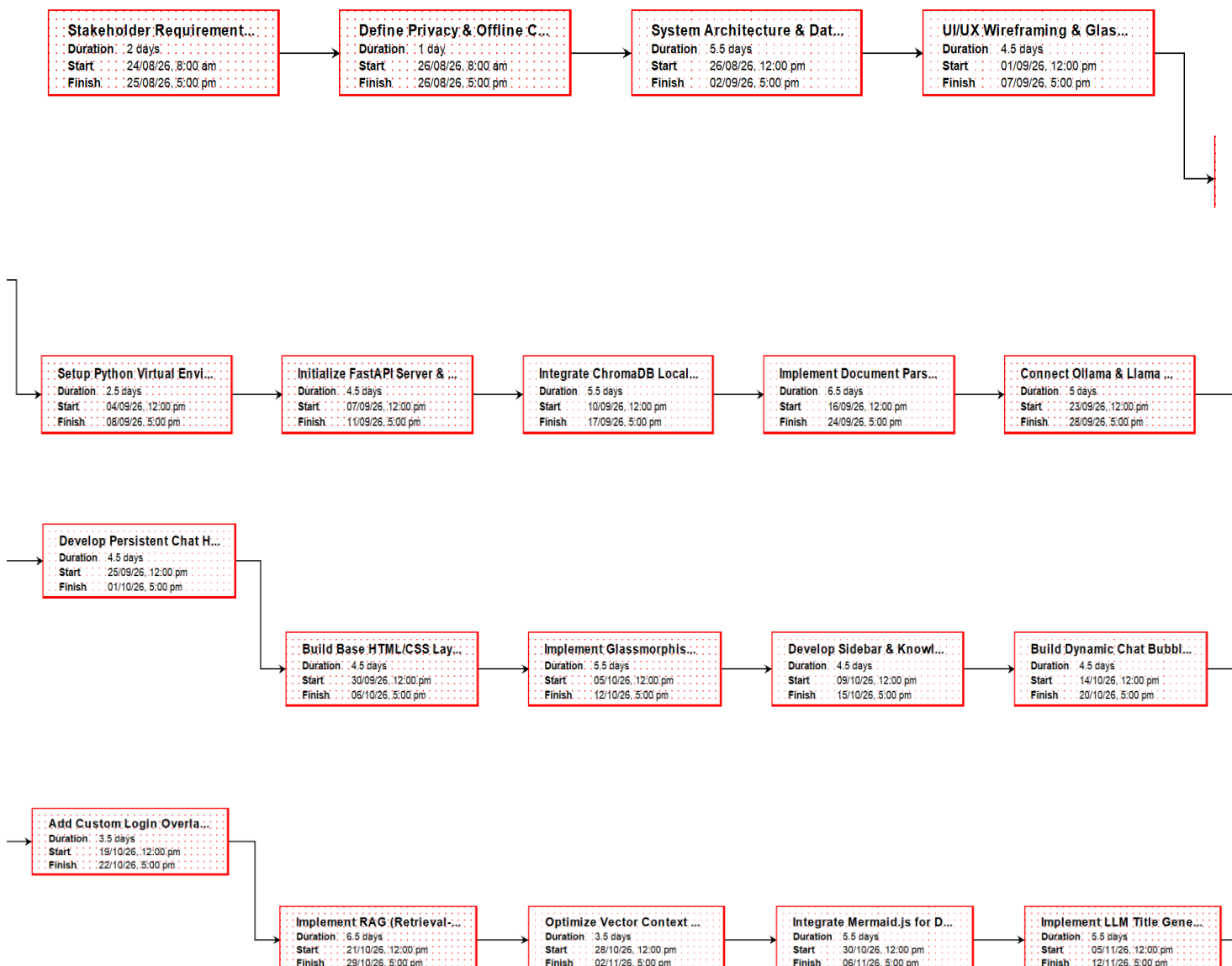


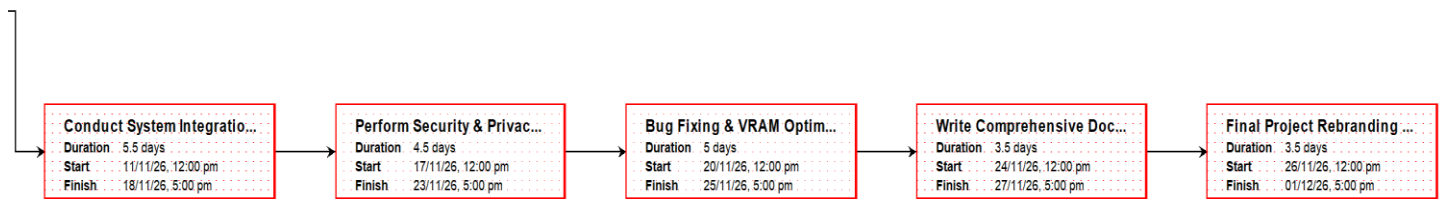
Fig2.1 Sample Diagram for Activity-on-node

Network Diagram:

The Network diagram graphically outlines the timeline and phases of the AirGap AI system development project. It visually represents the sequence of events and how each technical milestone acts as a prerequisite for the next.

- It begins with **Requirements & System Design** (10 days) to define the exact scope, technical constraints, and business feasibility, ensuring that the strict offline-only requirement is fundamentally built into the architecture.
- The **Backend & Core Infrastructure** phase (20 days) follows immediately, focusing on setting up Fast API, initializing Chroma DB, parsing documents, and connecting the local LLMs.
- **Frontend Development** (15 days) relies on the backend architecture being defined, focusing on the HTML/CSS glass morphism system and the dynamic chat interface.
- **Advanced AI Features** (15 days) takes the established frontend and backend and bridges them with advanced RAG logic and flowchart generation.
- The **Quality Assurance & Deployment** phase (15 days) handles final testing, the critical security audit, optimizing hardware usage, and finalizing the codebase.





Earliest start (ES) and earliest finish (EF) times, the latest start (LS) and latest finish (LF) times:

The earliest start and earliest finish are calculated by using the forward pass approach whereas for calculating latest start and latest finish, the backward pass approach is used.

1. Forward Pass:

- Start from the beginning and calculate the earliest start (ES) and earliest finish (EF) times for each activity.
- Use the formula:
 - $EF = ES + \text{Duration}$
 - ES for successors = EF of predecessor

2. Backward Pass:

- Start from the end and calculate the latest start (LS) and latest finish (LF) times for each activity.
- Use the formula:
 - $LS = LF - \text{Duration}$
 - LF for predecessors = LS of successor

Task Information

ID	Name	Start	Finish	Critical	Late Start	Late Finish	Free Slack	Total Slack
1	LocalLens AI Project	24/08/26, 8:00 am	04/12/26, 5:00 pm	true	24/08/26, 8:00 am	04/12/26, 5:00 pm	0 days	0 days
2	1. Requirements & System	24/08/26, 8:00 am	04/09/26, 5:00 pm	true	07/08/26, 8:00 am	21/08/26, 5:00 pm	-10 days	-10 days
3	Stakeholder Requirements	24/08/26, 8:00 am	25/08/26, 5:00 pm	true	07/08/26, 8:00 am	10/08/26, 5:00 pm	-11 days	-11 days
4	Define Privacy & Offline	26/08/26, 8:00 am	26/08/26, 5:00 pm	true	11/08/26, 8:00 am	11/08/26, 5:00 pm	-11 days	-11 days
5	System Architecture & Data Flow	26/08/26, 12:00 pm	01/09/26, 5:00 pm	true	12/08/26, 8:00 am	18/08/26, 12:00 pm	-10.5 days	-10.5 days
6	UI/UX Wireframing &	01/09/26, 12:00 pm	04/09/26, 5:00 pm	true	18/08/26, 12:00 pm	21/08/26, 5:00 pm	-10 days	-10 days
7	2. Backend & Core Infrastructure	04/09/26, 12:00 pm	30/09/26, 5:00 pm	true	24/08/26, 8:00 am	21/09/26, 5:00 pm	-7 days	-7 days
8	Setup Python Virtual	04/09/26, 12:00 pm	07/09/26, 5:00 pm	true	24/08/26, 8:00 am	25/08/26, 12:00 pm	-9.5 days	-9.5 days
9	Initialize FastAPI Server &	07/09/26, 12:00 pm	10/09/26, 5:00 pm	true	25/08/26, 12:00 pm	28/08/26, 5:00 pm	-9 days	-9 days
10	Integrate ChromaDB Local	10/09/26, 12:00 pm	16/09/26, 5:00 pm	true	31/08/26, 8:00 am	04/09/26, 12:00 pm	-8.5 days	-8.5 days
11	Implement Document Parsing	16/09/26, 12:00 pm	23/09/26, 5:00 pm	true	04/09/26, 12:00 pm	11/09/26, 5:00 pm	-8 days	-8 days
12	Connect Ollama & Llama 3.2	23/09/26, 12:00 pm	25/09/26, 5:00 pm	true	14/09/26, 8:00 am	16/09/26, 12:00 pm	-7.5 days	-7.5 days
13	Develop Persistent Chat History	25/09/26, 12:00 pm	30/09/26, 5:00 pm	true	16/09/26, 12:00 pm	21/09/26, 5:00 pm	-7 days	-7 days
14	3. Frontend Development	30/09/26, 12:00 pm	21/10/26, 5:00 pm	true	22/09/26, 8:00 am	15/10/26, 12:00 pm	-4.5 days	-4.5 days
15	Build Base HTML/CSS Layout	30/09/26, 12:00 pm	05/10/26, 5:00 pm	true	22/09/26, 8:00 am	25/09/26, 12:00 pm	-6.5 days	-6.5 days
16	Implement Glassmorphism	05/10/26, 12:00 pm	09/10/26, 5:00 pm	true	25/09/26, 12:00 pm	01/10/26, 5:00 pm	-6 days	-6 days
17	Develop Sidebar & Knowledge	09/10/26, 12:00 pm	14/10/26, 5:00 pm	true	02/10/26, 8:00 am	07/10/26, 12:00 pm	-5.5 days	-5.5 days
18	Build Dynamic Chat Bubble	14/10/26, 12:00 pm	19/10/26, 5:00 pm	true	07/10/26, 12:00 pm	12/10/26, 5:00 pm	-5 days	-5 days
19	Add Custom Login Overlay &	19/10/26, 12:00 pm	21/10/26, 5:00 pm	true	13/10/26, 8:00 am	15/10/26, 12:00 pm	-4.5 days	-4.5 days
20	4. Advanced AI Features	21/10/26, 12:00 pm	11/11/26, 5:00 pm	true	15/10/26, 12:00 pm	09/11/26, 12:00 pm	-2.5 days	-2.5 days
21	Implement RAG (Retrieval-	21/10/26, 12:00 pm	28/10/26, 5:00 pm	true	15/10/26, 12:00 pm	22/10/26, 5:00 pm	-4 days	-4 days
22	Optimize Vector Context Window	28/10/26, 12:00 pm	30/10/26, 5:00 pm	true	23/10/26, 8:00 am	27/10/26, 12:00 pm	-3.5 days	-3.5 days
23	Integrate Mermaid.js for Dynamic	30/10/26, 12:00 pm	05/11/26, 5:00 pm	true	27/10/26, 12:00 pm	02/11/26, 5:00 pm	-3 days	-3 days
24	Implement LLM Title Generation	05/11/26, 12:00 pm	11/11/26, 5:00 pm	true	03/11/26, 8:00 am	09/11/26, 12:00 pm	-2.5 days	-2.5 days
25	5. Quality Assurance &	11/11/26, 12:00 pm	30/11/26, 5:00 pm	true	09/11/26, 12:00 pm	30/11/26, 5:00 pm	0 days	0 days
26	Conduct System Integration	11/11/26, 12:00 pm	17/11/26, 5:00 pm	true	09/11/26, 12:00 pm	13/11/26, 5:00 pm	-2 days	-2 days
27	Perform Security & Privacy Audit	17/11/26, 12:00 pm	20/11/26, 5:00 pm	true	16/11/26, 8:00 am	19/11/26, 12:00 pm	-1.5 days	-1.5 days
28	Bug Fixing & VRAM Optimization	20/11/26, 12:00 pm	24/11/26, 5:00 pm	true	19/11/26, 12:00 pm	23/11/26, 5:00 pm	-1 day	-1 day
29	Write Comprehensive	24/11/26, 12:00 pm	26/11/26, 5:00 pm	true	24/11/26, 8:00 am	26/11/26, 12:00 pm	-0.5 days	-0.5 days
30	Final Project Rebranding & Git	26/11/26, 12:00 pm	30/11/26, 5:00 pm	true	26/11/26, 12:00 pm	30/11/26, 5:00 pm	0 days	0 days

Critical Path and Duration

1. Critical Path: The critical path is the longest sequence of dependent tasks in a project that determines the shortest possible time to complete the project. Tasks on this path have zero slack (or float), meaning any delay in these critical tasks will directly delay the entire project deadline. For AirGap AI, because the phases are heavily dependent on the completion of the previous technical milestone, the majority of the project lies on the critical path.

Steps to Find Critical Path:

- Find earliest start, earliest finish, latest finish, latest end.
- Identify the path with zero slack ($\text{Slack} = \text{LS} - \text{ES}$ or $\text{LF} - \text{EF}$).
- Calculate the total project duration based on the critical path.

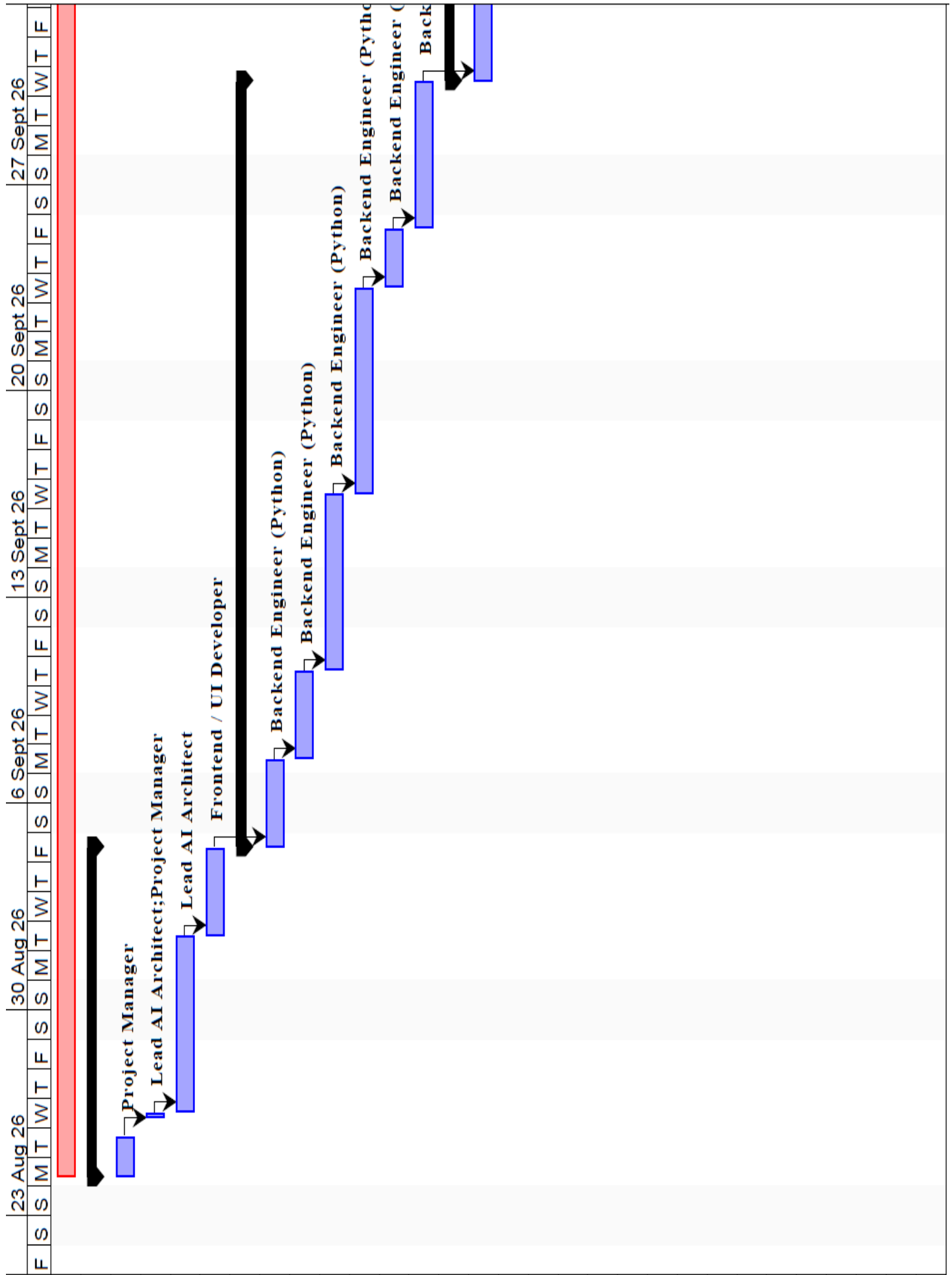
Total Project Duration: 75 days.

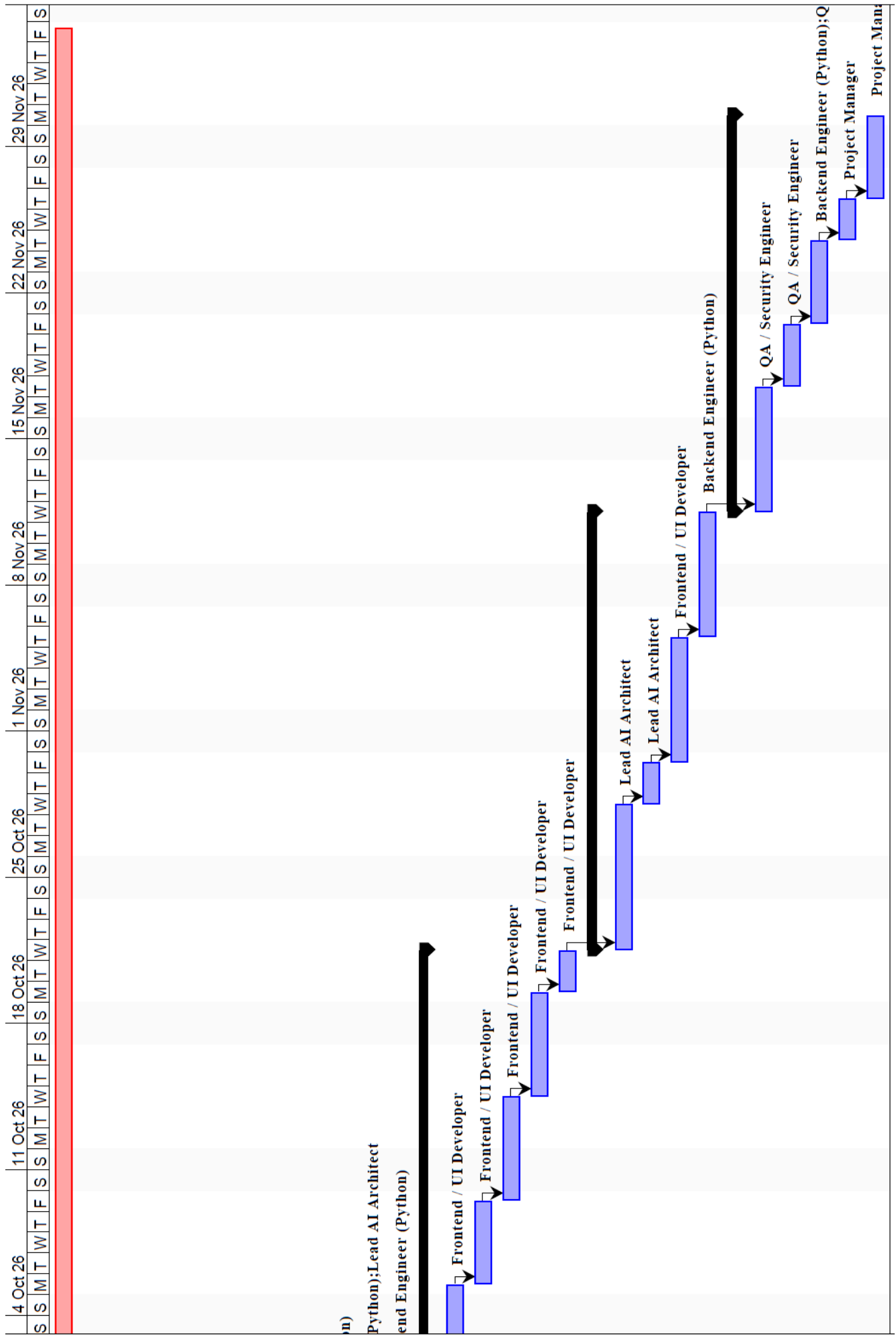
Gantt chart:

A Gantt chart is an essential project management tool that visually represents tasks, their specific durations, and their dependencies across a calendar timeline. It shows the planned start and finish dates of various sub-tasks. Tasks are displayed as horizontal bars, with the length of the bar directly corresponding to the task's duration. Gantt charts help the Air Gap AI team track daily progress, quickly identify potential delays, and ensure that tasks are completed strictly on schedule.

Steps:

- Use Project Libre to generate the Gantt chart based on the network diagram.
- Highlight the tasks on the critical path (shown in red), which are the ones that directly impact the project's completion date.





Part 3: Scheduling Resources and Costs

1. Resource Allocation

A resource histogram is a graphical representation of resource utilization over time. This shows exactly how human resources (like the Lead AI Architect or Backend Engineer) are distributed across the project timeline to prevent burnout.

Steps:

- Categorize resources into human resources, equipment, and materials.
- Create a histogram that plots resource usage over time to check for overallocation.

2. Types of Resources

Resources are the fundamental inputs needed for project completion. For the AirGap AI project, they are categorized as follows:

Human Resources

- **Project Manager:** Manages the overall project scope, gathers critical requirements, ensures privacy constraints are met, and coordinates all technical stakeholders.
- **Lead AI Architect:** Designs the overarching system architecture, oversees the implementation of the complex RAG pipeline, and integrates the local LLM APIs.
- **Backend Engineer (Python):** Designs and implements the backend logic, sets up the FastAPI server, manages the Chroma DB vector storage, and handles document parsing algorithms.
- **Frontend UI Developer:** Creates wireframes, mockups, and ensures the implementation of the complex glass morphism UI/UX design aligns perfectly with backend endpoints.
- **QA / Security Engineer:** Conducts rigorous unit, integration, and system testing. Specifically handles the strict Air Gap privacy audits to ensure zero external network leaks, and optimizes VRAM usage.

Software Resources

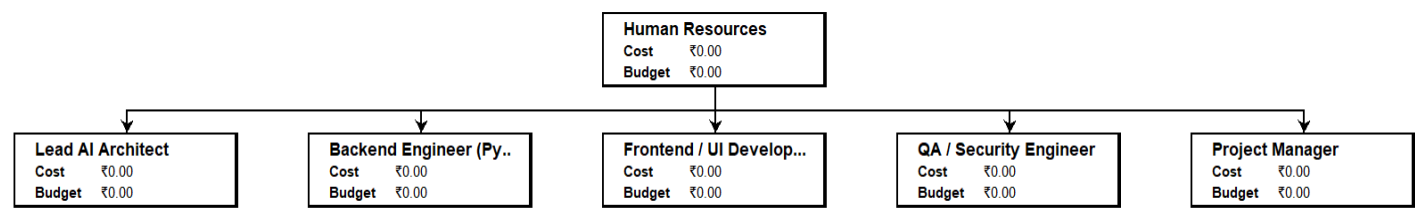
- **VSCode:** The primary code editor utilized for all Python and HTML/JS development.
- **Ollama:** The local LLM runner used to execute Llama 3.2 locally.
- **Chroma DB:** The software vector database used for local knowledge storage.
- **Git / GitHub:** Version Control System used to manage code versioning and team collaboration.

Hardware/Material Resources

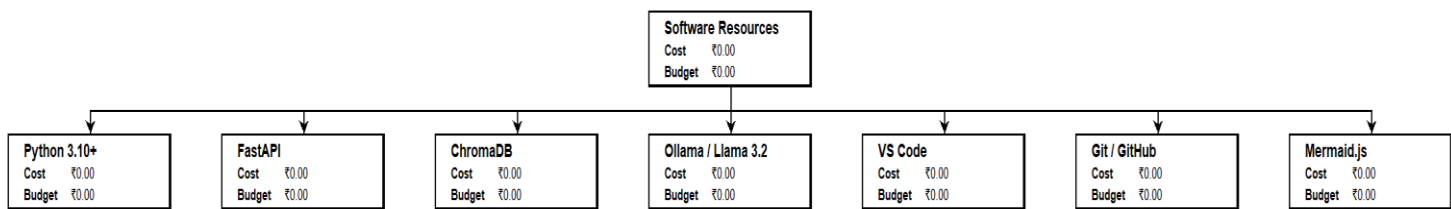
- **Local Workstation:** High-performance local machines required for testing.
- **Dedicated GPU:** Graphics Processing Unit (e.g., RTX 3060+) required to run local inference offline effectively.
- **Cloud Server GPU Time:** External compute power rented temporarily for training or heavy compilation tasks.

RBS for Human Resources

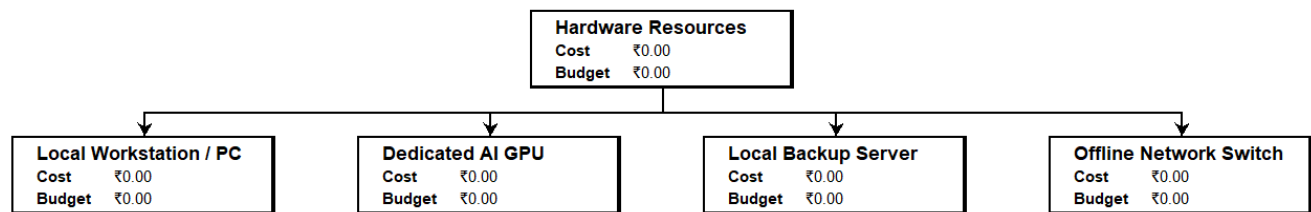
The Resource Breakdown Structure (RBS) graphically maps out the hierarchical reporting structure of the team.



RBS for Software Tools



RBS for Hardware Resources



Histograms:

1. Backend Engineer Utilization: The Backend Engineer plays a pivotal technical role with responsibilities that include developing the API, setting up databases, and parsing complex documents. Their involvement is massive during the second phase of the project, as illustrated in the following histogram. Over-allocations (spikes over 100%) must be carefully monitored during the database schema design phase.

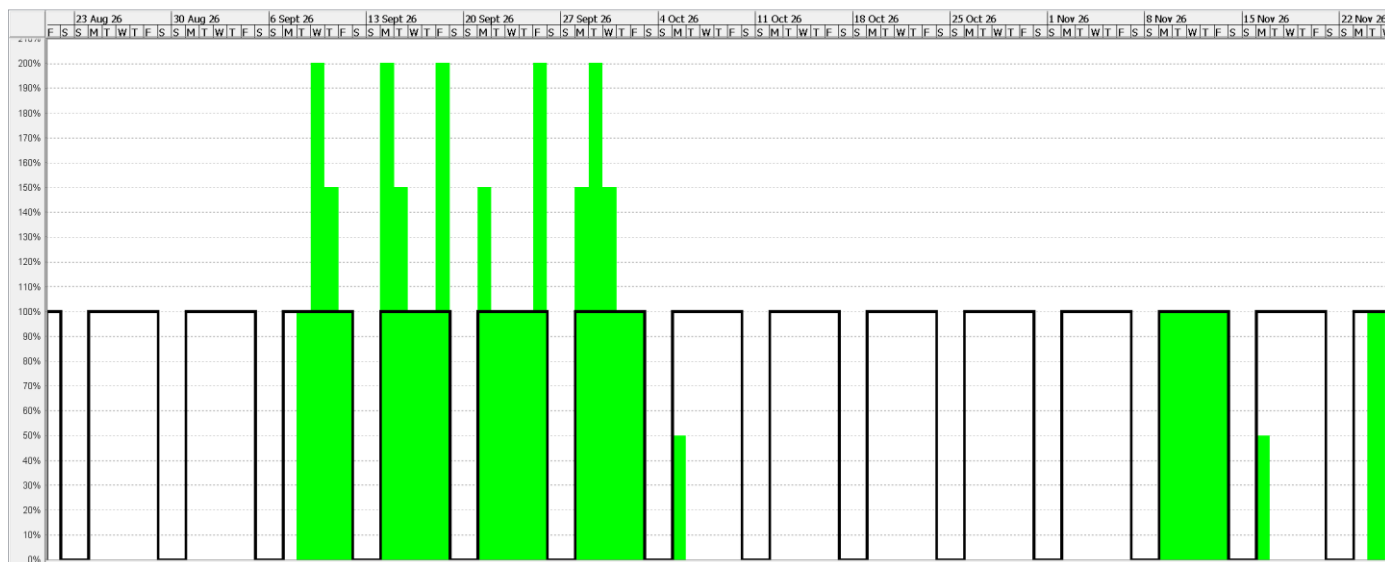


Fig3.1 Resource Utilization for Backend Engineer

2. Frontend UI Developer Utilization: The Frontend Developer is responsible for bringing the visual identity of Air Gap AI to life. Their workload peaks heavily during the third phase of the project when the HTML layout and glass morphism systems are being coded, requiring dedicated, uninterrupted focus.

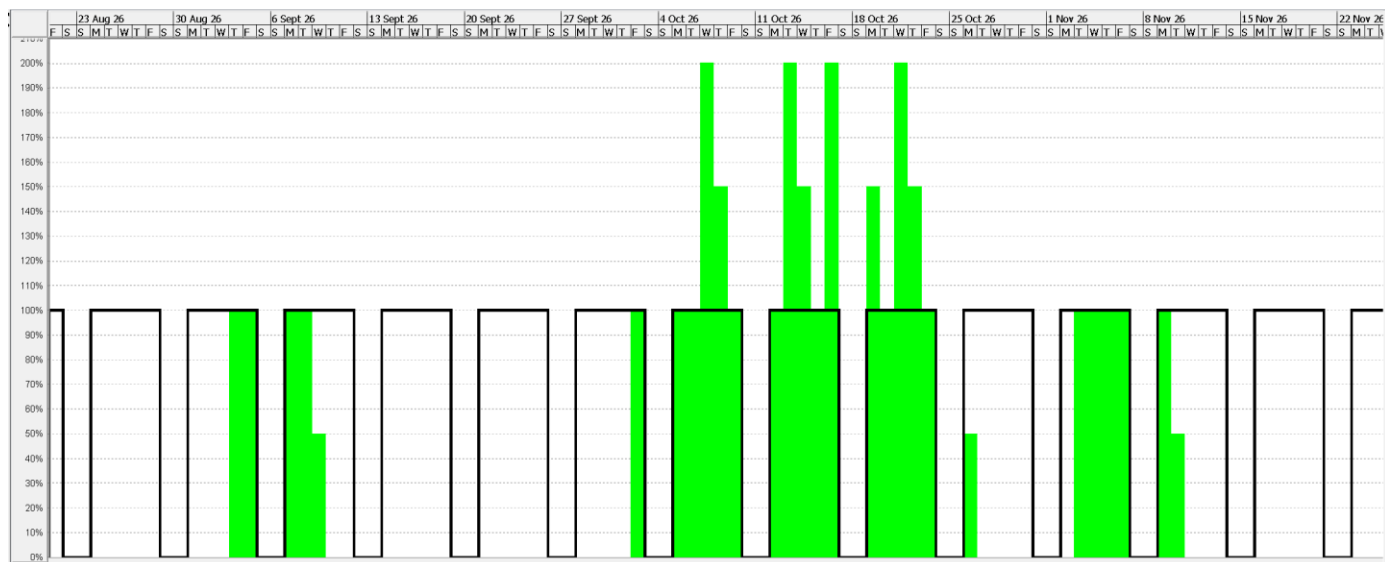


Fig3.2 Resource Utilization for UI/UX Designer

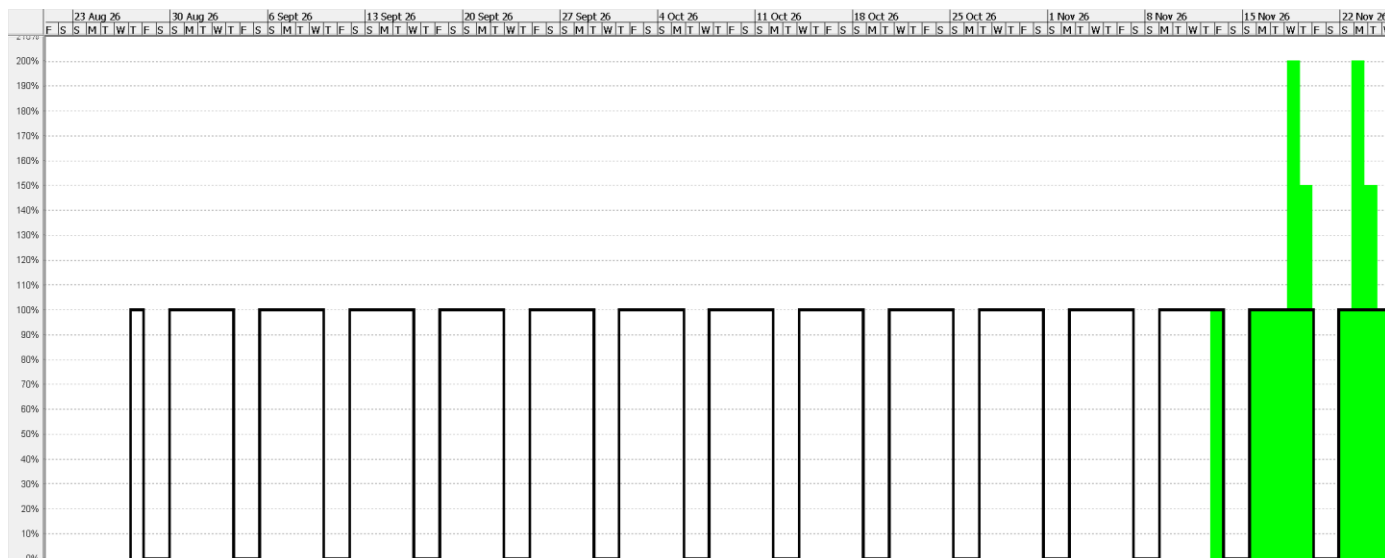


Fig3.3 Resource Utilization for QA/Security Engineer

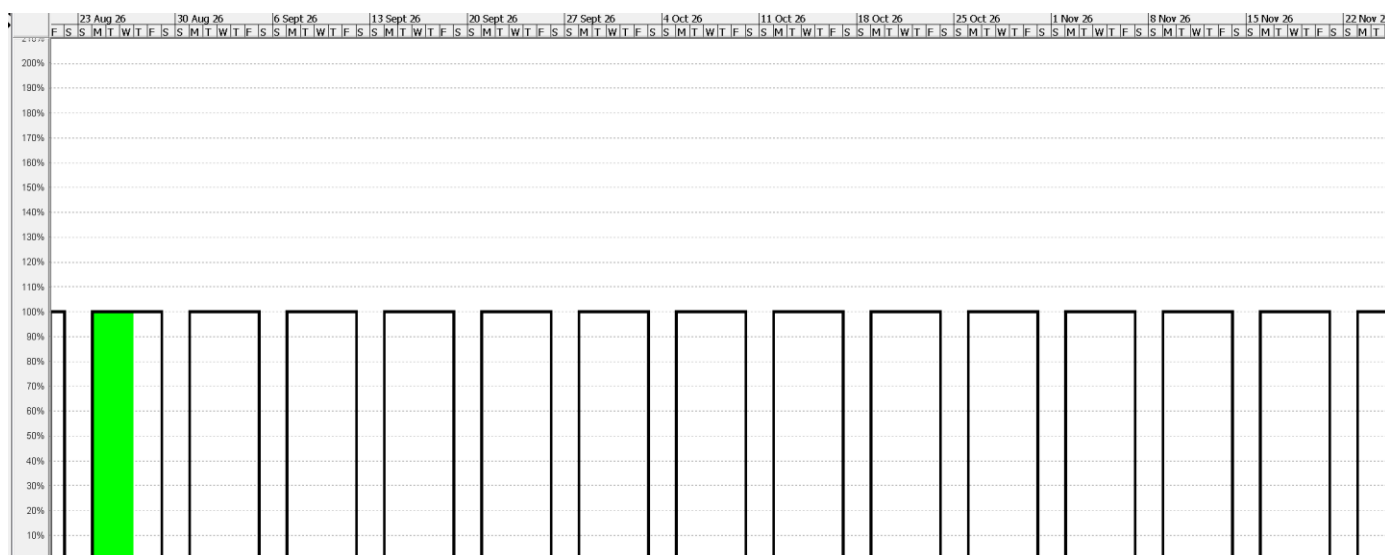


Fig3.4 Resource Utilization for Project Manager

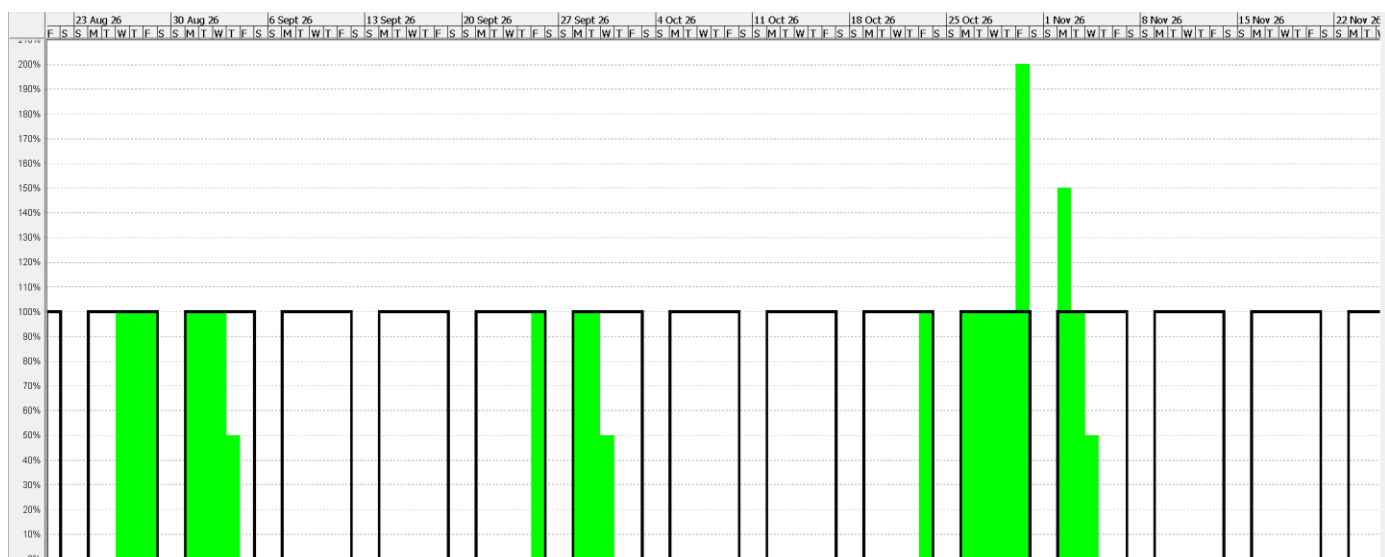


Fig3.5 Resource Utilization for Lead AI Architect

Part 4: Reducing Project Duration

1. Schedule Compression Techniques

Schedule compression involves advanced project management techniques like fast tracking and crashing to deliberately shorten the project duration without altering the fundamental project scope.

- **Fast Tracking:** Performing tasks concurrently that were originally planned sequentially.
- **Crashing:** Adding more resources (money or people) to tasks on the critical path to speed them up.

Steps:

1. Identify tasks specifically on the critical path where schedule compression is logically possible.
2. Apply the fast tracking and crashing techniques and recalculate the new, shortened project duration.

2. New Project Duration

Key Changes Made: 1. Task Rescheduling:

- Identified tasks in the Frontend and Backend phases that could be logically overlapped without introducing major risks to the project deliverables (e.g., agreeing on an API contract early).
- Adjusted dependencies between tasks from traditional "Finish-to-Start" to "Start-to-Start" or "Finish-to-Finish" where feasible.

2. Critical Path Analysis:

- Analyzed the critical path to identify tasks that directly impact the project timeline.
- Applied fast-tracking strictly to critical path activities to ensure the time reduction actually affected the final deadline.

3. Risk Assessment:

- Evaluated potential risks of overlapping complex tasks, such as increased chances of rework or miscommunication between the AI Architect and the Frontend Developer.
- Mitigated these risks by improving daily communication and monitoring task dependencies closely.

4. Resource Allocation:

- Ensured that resources were actually available and not severely overburdened (preventing 200% overallocation spikes) during the newly overlapping tasks.
- Updated the resource calendar to accurately reflect any changes in task schedules.

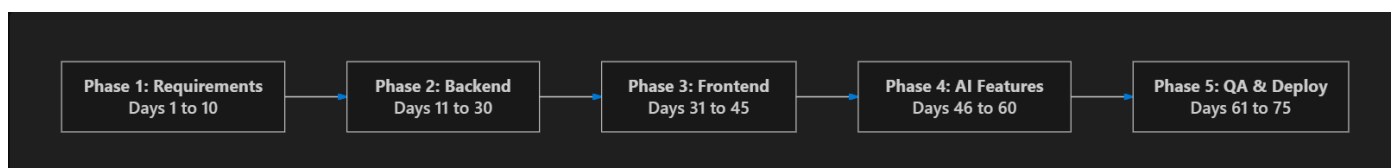
Benefits Achieved:

- **Reduced Project Duration:** The overall timeline was successfully compressed without compromising any major milestones, deliverables, or the strict offline privacy mandate.
- **Cost Efficiency:** Avoided massive additional costs by utilizing smart fast-tracking rather than immediately defaulting to expensive crashing techniques.
- **Improved Flexibility:** Enabled the engineering team to adapt to tight deadlines or changing client requirements smoothly.

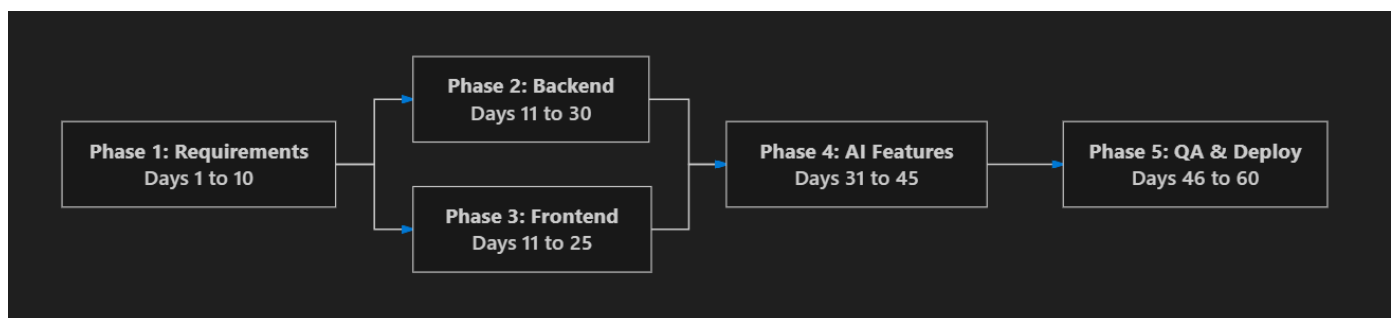
Fast Tracking

Fast tracking involves expediting the software development process by overlapping or executing specific steps, phases, or activities simultaneously to meet tight deadlines or achieve faster time-to-market. This approach is commonly used in agile, high-pressure AI environments where speed is prioritized. Following a detailed analysis of the Air Gap AI project planning activities, it was determined that the Frontend Development tasks could be executed concurrently alongside the Backend Infrastructure tasks, rather than waiting for the backend to be 100% complete. By scheduling these activities simultaneously and utilizing mock-APIs, the project timeline was successfully reduced by several days.

Before Fast Tracking



After Fast Tracking



Crashing

Crashing in software engineering, particularly in project management, refers to a technique used to forcefully shorten the project schedule by allocating additional financial or human resources to critical path activities. The primary goal of crashing is to reduce the project duration while mathematically minimizing the additional costs incurred.

Approach Used: To significantly speed up task completion during the highly complex "Advanced AI Features" phase, additional Python software developers were temporarily brought on board as contractors. This helped the core team work much faster and finish the RAG pipeline integration sooner. However, it also meant noticeably higher costs due to the extra hourly wages of the contractors. The challenge was to carefully weigh these increased financial costs against the immense value of completing the AI project in a shorter timeframe, ensuring it made financial sense for the stakeholders overall.

Before Crashing

	Name	Duration	Start	Finish	Predecessors
	LocalLens AI Project	75 days	24/08/26, 8:00 am	04/12/26, 5:00 pm	
	1. Requirements & System Design	11 days	24/08/26, 8:00 am	07/09/26, 5:00 pm	
	Stakeholder Requirements	2 days	24/08/26, 8:00 am	25/08/26, 5:00 pm	
	Define Privacy & Offline Capabilities	1 day	26/08/26, 8:00 am	26/08/26, 5:00 pm	3
	System Architecture & Data Flow	5.5 days	26/08/26, 12:00 pm	02/09/26, 5:00 pm	4
	UI/UX Wireframing & Glassmorphism	4.5 days	01/09/26, 12:00 pm	07/09/26, 5:00 pm	5
	2. Backend & Core Infrastructure	19.5 days	04/09/26, 12:00 pm	01/10/26, 5:00 pm	
	Setup Python Virtual Environment	2.5 days	04/09/26, 12:00 pm	08/09/26, 5:00 pm	6
	Initialize FastAPI Server & CORS	4.5 days	07/09/26, 12:00 pm	11/09/26, 5:00 pm	8
	Integrate ChromaDB Local Vector Store				
	Implement Document Parser (LangChain)				
	Connect Ollama & Llama 3.1 Inference				
	Develop Persistent Chat History				
	3. Frontend Development				
	Build Base HTML/CSS Layout				
	Implement Glassmorphism Effects				
	Develop Sidebar & Knowledge Base				
	Build Dynamic Chat Bubbles				
	Add Custom Login Overlay				
	4. Advanced AI Features				
	Implement RAG (Retrieval-Augmented Generation)				
	Optimize Vector Context Window				
	Integrate Mermaid.js for Diagrams				
	Implement LLM Title Generation				
	5. Quality Assurance & Deployment				
	Conduct System Integration Testing				
	Perform Security & Privacy Audit				
	Bug Fixing & VRAM Optimization				
	Write Comprehensive Documentation				
	Final Project Rebranding & Launch	3.5 days	26/11/26, 12:00 pm	01/12/26, 5:00 pm	29

Project AI Dev

Assign Resources

×

Tasks: 25

	Name	Units
	Lead AI Architect	
	Backend Engineer (Python)	
	Frontend / UI Developer	
	QA / Security Engineer	
	Project Manager	

Assign

Remove

Replace...

Help

After Crashing

Gantt
 Network
 WBS
 Task Usage

Zoom In
 Zoom Out

Copy
 Cut
 Paste

Insert
 Delete

Indent
 Outdent

Link
 Unlink

Information
 Calendar
 Notes

Assign
 Save Ba
 Clear Ba

	Name	Duration	Start	Finish	
1	LocalLens AI Project	75 days	24/08/26, 8:00 am	04/12/26, 5:00 pm	
2	1. Requirements & System Design	11 days	24/08/26, 8:00 am	07/09/26, 5:00 pm	
3	Stakeholder Requirements	2 days	24/08/26, 8:00 am	25/08/26, 5:00 pm	
4	Define Privacy & Offline Capabilities	1 day	26/08/26, 8:00 am	26/08/26, 5:00 pm	3
5	System Architecture & Data Flow	5.5 days	26/08/26, 12:00 pm	02/09/26, 5:00 pm	4
6	UI/UX Wireframing & Glassmorphism	4.5 days	01/09/26, 12:00 pm	07/09/26, 5:00 pm	5
7	2. Backend & Core Infrastructure	19.5 days	04/09/26, 12:00 pm	01/10/26, 5:00 pm	
8	Setup Python Virtual Environment	2.5 days	04/09/26, 12:00 pm	08/09/26, 5:00 pm	6
9	Initialize FastAPI Server & Dependencies	4.5 days	07/09/26, 12:00 pm	11/09/26, 5:00 pm	8
10	Integrate ChromaDB Local Vector Store	5.5 days	10/09/26, 12:00 pm	17/09/26, 5:00 pm	9
11	Implement Document Parser & Embedder	6.5 days	16/09/26, 12:00 pm	24/09/26, 5:00 pm	10
12	Connect Ollama & Llama 3.2 Model				
13	Develop Persistent Chat History				
14	3. Frontend Development	16 days			
15	Build Base HTML/CSS Layout				
16	Implement Glassmorphism Effects				
17	Develop Sidebar & Knowledge Base				
18	Build Dynamic Chat Bubbles				
19	Add Custom Login Overlay				
20	4. Advanced AI Features	16 days			
21	Implement RAG (Retrieval-Augmented Generation)				
22	Optimize Vector Context Window				
23	Integrate Mermaid.js for Diagrams				
24	Implement LLM Title Generation				
25	5. Quality Assurance & Deployment	14 days			
26	Conduct System Integration Testing				
27	Perform Security & Privacy Audit				
28	Bug Fixing & VRAM Optimization				
29	Write Comprehensive Documentation				
30	Final Project Rebranding & Launch				

Project Overview

Assign Resources

Tasks: 4

	Name	Units
☒	Lead AI Architect	200%
☐	Backend Engineer (Python)	
☐	Frontend / UI Developer	
☐	QA / Security Engineer	
☒	Project Manager	100%

☒
☒